

B: Binary Dozens

Source file name: `bindozens.c`, `bindozens.cpp`, `bindozens.java`, or `bindozens.py`

Author: Rodrigo Cardoso

Egan recently founded a nano-egg company (microscopic eggs with promising pharmaceutical uses). Because eggs are typically sold by the dozen, Egan also developed *nano-crates*, each holding exactly 12 nano-eggs. The company sells only full dozens: an integer number of nano-crates, each completely filled.

Egg production is costly. Given a stock of H eggs, let D be the number of eggs that do not fit into complete nano-crates (12 eggs each). These surplus eggs must be discarded.

The following examples show some possible cases:

- If $H = 33$, two crates are used, discarding $D = 9$ eggs.
- If $H = 48$, four crates are used, discarding $D = 0$ eggs.

Production volumes can be very large and the line uses a binary counter to record batch size. Thus, H in decimal can be expressed in *binary* as B . Then, the previous examples should be interpreted as follows:

- If $B = 100001$, then $D = 9$.
- If $B = 110000$, then $D = 0$.

Help Egan by writing a program that reads B and outputs D in decimal.

Input

The input consists of multiple test cases, one per line. Each test case is a binary string B (characters '0' and '1' only) representing the number of nano-eggs H . You may assume $1 \leq |B| \leq 500$.

The end of the input is signaled with a line containing a single '*'.

The input must be read from standard input.

Output

For each test case B , output the number D of leftover nano-eggs when packing in nano-crates.

The output must be written to standard output.

Sample Input	Sample Output
100001	9
110000	0
1	1
111	7
1010110	2
*	